59

BÖLÜM: MAANG EŞZAMANLILIK MÜLAKATI & SISTEM TASARIMI USTALIĞI | MODULE: MAANG CONCURRENCY WHITEBOARD & SYSTEM DESIGN MASTERY

Eşzamanlılık Beyaz Tahta Planı: 45 Dakikalık MAANG Mülakat Yönetim Stratejisi

Concurrency Whiteboard Blueprint: 45-Minute MAANG Execution Strategy

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
                45 DAKİKALIK MAANG CONCURRENCY MÜLAKAT ZAMAN ÇİZELGESİ

00-05 dk: GEREKSİNİMLERİ NETLEŞTİR (Clarify Requirements & Thread Safety Scope)
05-12 dk: VERİ YAPILARI & ÇEKİŞME ANALİZİ (Data Structures & Contention Points)
12-25 dk: ÇALIŞAN İLK KODU YAZ (Baseline Thread-Safe Code - Monolithic Lock)
25-35 dk: KİLİTSİZ / DETAYLI KİLİT OPTİMİZASYONU (Fine-Grained / CAS / Disruptor)
35-42 dk: SINIR DURUMLAR & TEŞHİS (Deadlock, Spurious Wakeups, Memory Leaks)
42-45 dk: ÖZET & STAFF+ MİMAR PERSPEKTİFİ (Metrics, Loom Virtual Threads, Tuning)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

MAANG mülakatı bir boks maçına benzer. Ringe çıkar çıkmaz gözün kapalı yumruk sallarsan (soruyu dinlemeden hemen kod yazmaya başlarsan) ilk rauntta nakavt olursun! Önce rakibini süz (soruyu sor: 'Kaç thread yazacak? Okuma mı çok yazma mı?'). Sonra gardını alıp sağlam bir vuruş yap (önce basit ama çalışan kilitli kodu yaz). Sonra özel hareketini göster (Lock-Free veya Virtual Thread optimizasyonunu açıkla). Hakemler şovuna hayran kalır!

🇬🇧 ENGLISH (ELI5)

A MAANG concurrency interview is like a surgical operation. If you grab the scalpel and start cutting before reviewing the patient's vitals (writing code without clarifying read/write contention ratios), the patient dies. First clarify invariants and boundaries. Second, provide a working baseline using clean abstractions. Third, optimize hot paths using fine-grained locks or lock-free CAS. Finally, walk through edge cases like thread starvation, spurious wakeups, and observability.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Mülakatçının Not Defteri ve 5 Kritik Değerlendirme Ekseni: 1) Soru Sorma (Requirement Clarification): 'Sistemimiz read-heavy mi yoksa write-heavy mi? Gecikme toleransı (SLA) nedir? Thread sayısı sabit mi yoksa dinamik mi?' sorularını sormak adayı anında Senior havuzuna taşır. 2) İlerlemeli Tasarım (Progressive Enhancement): Doğrudan aşırı karmaşık lock-free kod yazmaya çalışmayın! Önce `synchronized` veya `ReentrantLock` ile %100 doğru çalışan bir temel (baseline) kurun. Ardından 'Burada çekişme olursa Striped Locking veya ConcurrentHashMap'e evrilebiliriz' diyerek optimizasyona geçin. 3) Bellek Modeli Hissiyatı (JMM Fluency): Kod yazarken `volatile`, happens-before ve görünürlük etkilerini sesli olarak mülakatçıya anlatın. 4) Köşe Vakalar (Edge Cases): Spurious wakeups, interrupt durumunda thread bayrağını koruma (`Thread.currentThread().interrupt()`), ve kilitlerin `finally` içinde bırakılması adayın reflekslerini gösterir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Interviewer Scoring Rubric: 1) Clarification Phase: Proactively inquire about read-to-write ratios, thread counts, latency budgets, and memory constraints. 2) Progressive Implementation: Never dive prematurely into unverified lock-free wizardry. Implement an unassailable baseline using idiomatic locks first, then systematically profile and optimize hotspots to demonstrate maturity. 3) Mechanical & JMM Fluency: Articulate happens-before edges, CPU memory fences, and cache-line effects aloud while drafting variables. 4) Defensive Rigor: Always restore interrupted statuses, guarantee lock releases in `finally` blocks, and guard condition checks within `while` loops against spurious wakeups.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: MAANG Mülakatında Sunulacak İdeal Şablon Kodu
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

public class MaangInterviewTemplate<T> {
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition condition = lock.newCondition();
    private boolean ready = false;

    public void waitForSignal() throws InterruptedException {
        lock.lock();
        try {
            // ⚠ MÜLAKAT KURALI 1: if değil daima WHILE kullan (Spurious Wakeup Savunması!)
            while (!ready) {
                condition.await();
            }
            System.out.println("Sinyal alındı, kritik operasyon devam ediyor...");
        } finally {
            // ⚠ MÜLAKAT KURALI 2: Kilit DAİMA finally içinde açılır!
            lock.unlock();
        }
    }

    public void sendSignal() {
        lock.lock();
        try {
            ready = true;
            condition.signalAll(); // Tüm bekleyenleri haberdar et
        } finally {
            lock.unlock();
        }
    }

    void main() {
        System.out.println("MAANG şablonu hazır.");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakatçının 'Neden synchronized yerine ReentrantLock kullandın?' sorusuna hazırlıklı olun. Yanıtınız: tryLock (timeout), adil kilit seçeneği, çoklu Condition kuyruğu ve kesilebilir (lockInterruptibly) kilitleme yetenekleri olmalıdır.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Be prepared for the inevitable question: 'Why prefer ReentrantLock over synchronized?' Your crisp answer: timed polling via `tryLock()`, interruptible lock acquisition, fairness flags, and multiple independent `Condition` wait-queues.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

🎯 Key Takeaways for the Interview

- Mülakata önce gereksinimleri netleştirerek başlayın. Basit ve doğru bir kilitli temel kurun, ardından lock-free optimizasyonları ve köşe vakaları tartışın.

- Begin by clarifying contention ratios. Implement a robust baseline first, then progressively optimize with lock-free primitives and edge-case defenses.